

---

---

# Session 06 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

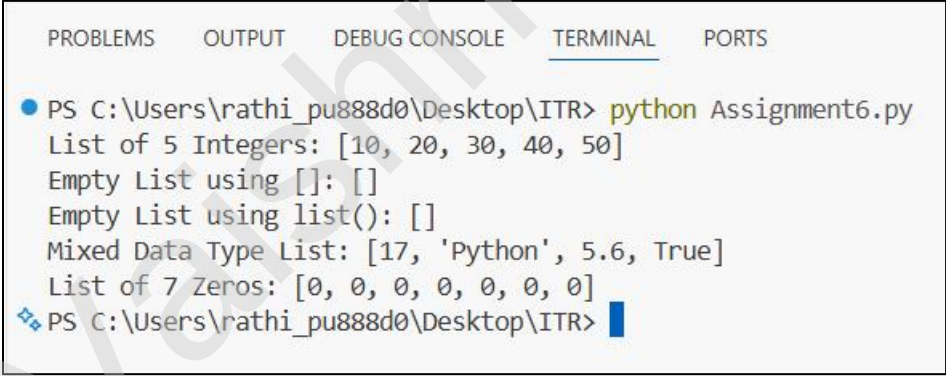
Domain: **AIML (Artificial Intelligence and Machine Learning)**

---

Q1. Creating Lists

---

**Output Screenshot:**



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
List of 5 Integers: [10, 20, 30, 40, 50]
Empty List using []: []
Empty List using list(): []
Mixed Data Type List: [17, 'Python', 5.6, True]
List of 7 Zeros: [0, 0, 0, 0, 0, 0, 0]
❖ PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

---

## Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q1. (Creating Lists)

Write a program to create the following lists using different methods:

- a) A list of 5 integers directly.
- b) An empty list using both [] and list().
- c) A list containing mixed data types (int, string, float, boolean).
- d) A list of 7 zeros using repetition (\*).

Print all lists with clear labels and add comments explaining each creation method.

```
'''
```

```
# Creating a list of 5 integers directly
```

```
integer_list = [10, 20, 30, 40, 50]
```

```
# Creating an empty list using square brackets
```

```
empty_list1 = []
```

```
# Creating an empty list using the list() constructor
```

```
empty_list2 = list()
```

```
# Creating a list with mixed data types
```

```
mixed_list = [17, "Python", 5.6, True]
```

```
# Creating a list of 7 zeros using repetition operator (*)
```

```
zero_list = [0] * 7
```

```
# Printing all lists with labels
```

```
print("List of 5 Integers:", integer_list)
```

```
print("Empty List using []:", empty_list1)
```

```
print("Empty List using list():", empty_list2)
```

```
print("Mixed Data Type List:", mixed_list)
```

```
print("List of 7 Zeros:", zero_list)
```

```
"""Explanation in Comments"""
```

```
# integer_list is created directly by placing elements inside square brackets.
```

```
# empty_list1 is an empty list created using [].
```

```
# empty_list2 is an empty list created using the list() constructor.
```

```
# mixed_list contains different data types such as int, string, float, and boolean.
```

```
# zero_list uses the repetition operator (*) to create seven zeros.
```

```
# Lists can store multiple values and different types of data.
```

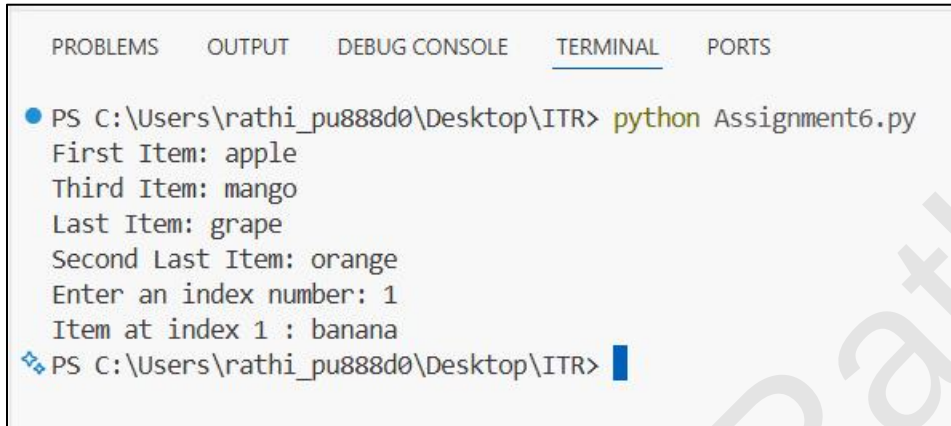
---

---

## Q2. Indexing & Negative Indexing

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
First Item: apple
Third Item: mango
Last Item: grape
Second Last Item: orange
Enter an index number: 1
Item at index 1 : banana
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

""" ASSIGNMENT 6 """

'''

Q2. (Indexing and Negative Indexing)

Create a list:

fruits = ["apple", "banana", "mango", "orange", "grape"]

Write code to print:

First item

Third item

Last item (using negative index)

Second last item (using negative index)

Take a number as input from the user and print the item at that index  
(handle basic cases only).

'''

# Creating the list

fruits = ["apple", "banana", "mango", "orange", "grape"]

```
# Printing required items
print("First Item:", fruits[0])
print("Third Item:", fruits[2])
print("Last Item:", fruits[-1])
print("Second Last Item:", fruits[-2])

# Taking index input from the user
index = int(input("Enter an index number: "))

# Basic index validation
if 0 <= index < len(fruits):
    print("Item at index", index, ":", fruits[index])
else:
    print("Invalid Index")

"Explanation in Comments"

# Positive indexing starts from 0 and moves from left to right.
# fruits[0] gives the first item.
# fruits[2] gives the third item.

# Negative indexing starts from -1 and moves from right to left.
# fruits[-1] gives the last item.
# fruits[-2] gives the second last item.

# len(fruits) returns the total number of items in the list.
# The if condition checks whether the entered index is valid.
```

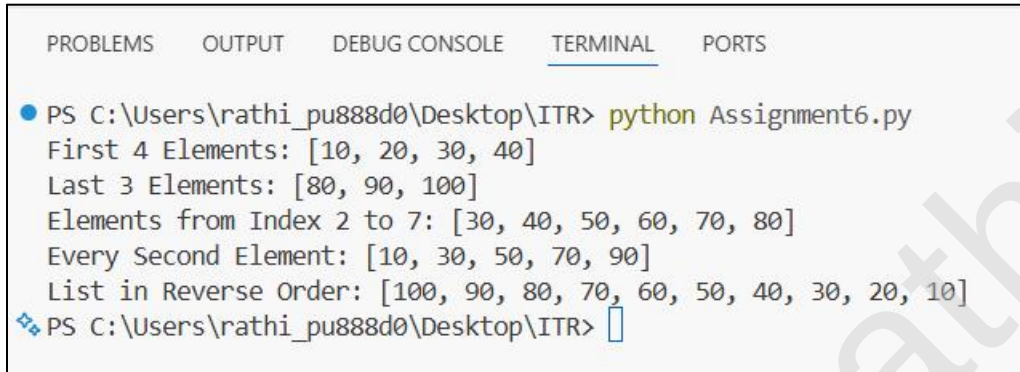
---

---

### Q3. Slicing Lists

---

#### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
First 4 Elements: [10, 20, 30, 40]
Last 3 Elements: [80, 90, 100]
Elements from Index 2 to 7: [30, 40, 50, 60, 70, 80]
Every Second Element: [10, 30, 50, 70, 90]
List in Reverse Order: [100, 90, 80, 70, 60, 50, 40, 30, 20, 10]
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

---

#### Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q3. (Slicing Lists)

Given the list:

```
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

Using slicing, print:

- a) First 4 elements
- b) Last 3 elements
- c) Elements from index 2 to 7
- d) Every second element
- e) The entire list in reverse order

Add comments explaining slicing syntax.

```
'''
```

```
# Creating the list
```

```
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

```
# a) First 4 elements
```

```
print("First 4 Elements:", numbers[:4])
```

```
# b) Last 3 elements
```

```
print("Last 3 Elements:", numbers[-3:])
```

```
# c) Elements from index 2 to 7
print("Elements from Index 2 to 7:", numbers[2:8])
```

```
# d) Every second element
print("Every Second Element:", numbers[::2])
```

```
# e) Entire list in reverse order
print("List in Reverse Order:", numbers[::-1])
```

**"Explanation in Comments"**

```
# List slicing uses the syntax [start:end:step].
```

```
# start -> Starting index (inclusive).
```

```
# end -> Ending index (exclusive).
```

```
# step -> Number of positions to move.
```

```
# numbers[:4] returns elements from index 0 to 3.
```

```
# numbers[-3:] returns the last 3 elements.
```

```
# numbers[2:8] returns elements from index 2 to 7.
```

```
# numbers[::2] returns every second element.
```

```
# numbers[::-1] reverses the list.
```

```
# Slicing helps extract specific portions of a list easily.
```

.

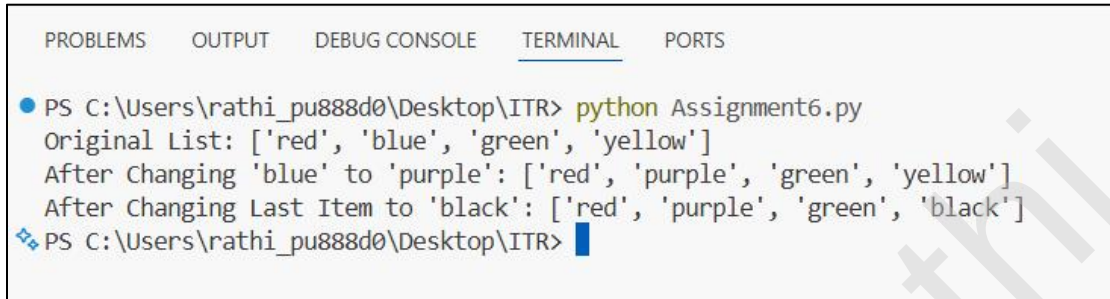
---

---

## Q4. Modifying Lists

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
Original List: ['red', 'blue', 'green', 'yellow']
After Changing 'blue' to 'purple': ['red', 'purple', 'green', 'yellow']
After Changing Last Item to 'black': ['red', 'purple', 'green', 'black']
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

---

### Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q4. (Modifying Lists)

Create a list:

```
colors = ["red", "blue", "green", "yellow"]
```

Perform the following:

1. Change "blue" to "purple" using indexing.
2. Change the last item to "black".
3. Print the list after each change.

```
'''
```

```
# Creating the list
```

```
colors = ["red", "blue", "green", "yellow"]
```

```
print("Original List:", colors)
```

```
# Change "blue" to "purple"
```

```
colors[1] = "purple"
```

```
print("After Changing 'blue' to 'purple':", colors)
```

```
# Change the last item to "black"
```

```
colors[-1] = "black"
```

```
print("After Changing Last Item to 'black':", colors)
```

**"Explanation in Comments"**

# Lists are mutable, which means their elements can be modified.  
# colors[1] accesses the second element and changes its value.  
# colors[-1] accesses the last element using negative indexing.  
# The list is printed after each modification to show the changes.

---

Vaishnavi Rathni

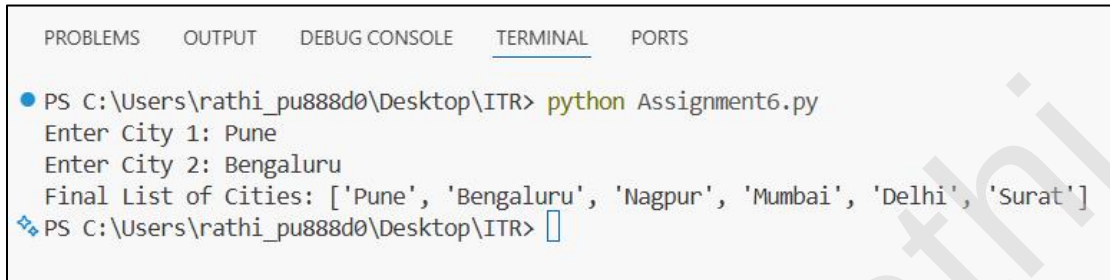


---

Q5. append() and insert()

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
Enter City 1: Pune
Enter City 2: Bengaluru
Final List of Cities: ['Pune', 'Bengaluru', 'Nagpur', 'Mumbai', 'Delhi', 'Surat']
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

### Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q5. (append() and insert())

Write a program that starts with an empty list.

Use append() to add 5 different city names one by one.

Use insert() to add one more city at position 2.

Print the final list.

Take user input for at least 2 cities.

```
'''
```

```
# Create an empty list
```

```
cities = []
```

```
# Add city names using append()
```

```
city1 = input("Enter City 1: ")
```

```
city2 = input("Enter City 2: ")
```

```
cities.append(city1)
```

```
cities.append(city2)
```

```
cities.append("Mumbai")
```

```
cities.append("Delhi")
```

```
cities.append("Surat")
```

```
# Add a city at position 2 using insert()
```

```
cities.insert(2, "Nagpur")
```

```
# Print the final list  
print("Final List of Cities:", cities)
```

**"Explanation in Comments"**

```
# append() adds an element at the end of the list.  
# insert(index, value) adds an element at a specific position.  
# The list starts empty and cities are added one by one.  
# Two city names are taken from the user as input.  
# The final list is displayed after all insertions.
```

---

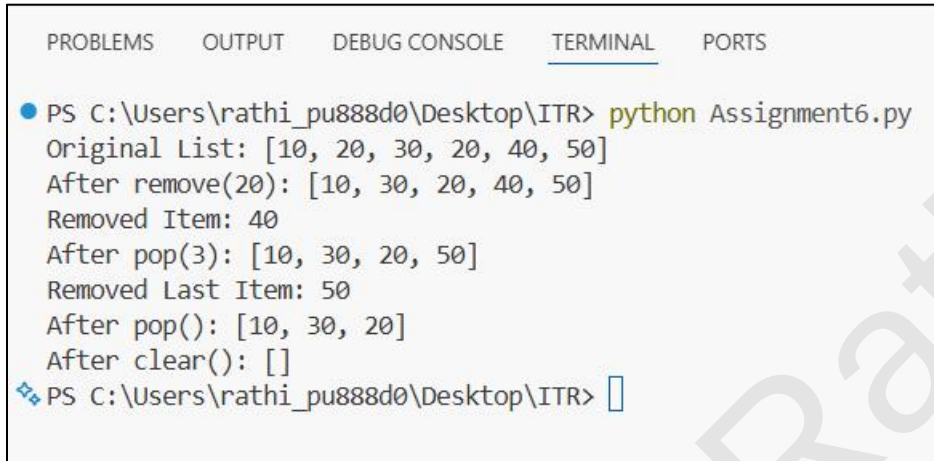
Vaishnavi Rathni

---

Q6. remove(), pop() and clear()

---

**Output Screenshot:**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
Original List: [10, 20, 30, 20, 40, 50]
After remove(20): [10, 30, 20, 40, 50]
Removed Item: 40
After pop(3): [10, 30, 20, 50]
Removed Last Item: 50
After pop(): [10, 30, 20]
After clear(): []
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

---

**Program Code:**

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q6. (remove(), pop(), and clear())

Create a list:

```
items = [10, 20, 30, 20, 40, 50]
```

Perform and print after each operation:

- a) Remove the first occurrence of 20 using remove().
- b) Remove the item at index 3 using pop() and print the removed value.
- c) Remove the last item using pop().
- d) Clear the entire list using clear().

Add comments explaining the difference between remove() and pop().

```
'''
```

```
# Creating the list
```

```
items = [10, 20, 30, 20, 40, 50]
```

```
print("Original List:", items)
```

```
# a) Remove the first occurrence of 20
```

```
items.remove(20)
```

```
print("After remove(20):", items)
```

```
# b) Remove the item at index 3
removed_item = items.pop(3)
print("Removed Item:", removed_item)
print("After pop(3):", items)
```

```
# c) Remove the last item
last_item = items.pop()
print("Removed Last Item:", last_item)
print("After pop():", items)
```

```
# d) Clear the entire list
items.clear()
print("After clear():", items)
```

"Explanation in Comments"

```
# remove(value) removes the first occurrence of the specified value.
# pop(index) removes an element at the given index and returns it.
# pop() without an index removes and returns the last element.
# clear() removes all elements from the list.
```

```
# Difference between remove() and pop():
# remove() deletes an element by value.
# pop() deletes an element by index and returns the removed element.
```

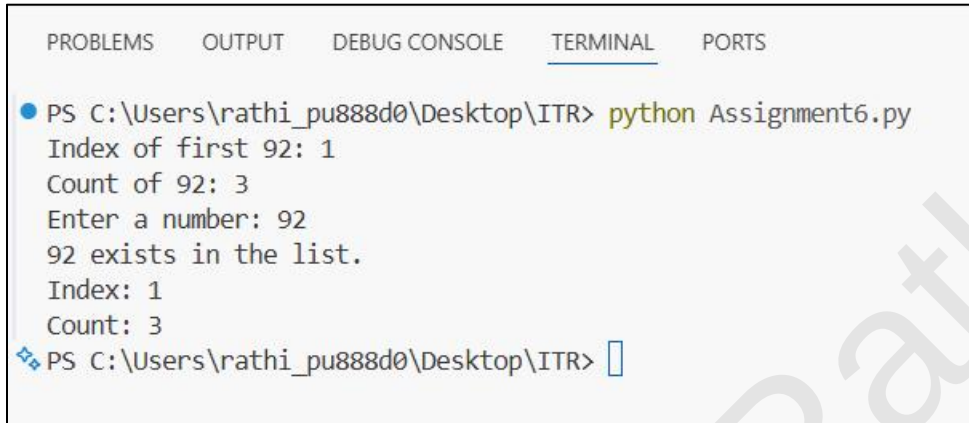
---

---

Q7. index() and count()

---

### Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu88d0\Desktop\ITR> python Assignment6.py
Index of first 92: 1
Count of 92: 3
Enter a number: 92
92 exists in the list.
Index: 1
Count: 3
PS C:\Users\rathi_pu88d0\Desktop\ITR> 
```

### Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q7. (index() and count())

Create a list:

```
scores = [85, 92, 78, 92, 65, 92, 88]
```

Find and print the index of the first occurrence of 92.

Count and print how many times 92 appears.

Take a number as input from the user and check if it exists in the list.

If yes, print its index and count.

```
'''
```

```
# Creating the list
```

```
scores = [85, 92, 78, 92, 65, 92, 88]
```

```
# Find the index of the first occurrence of 92
```

```
print("Index of first 92:", scores.index(92))
```

```
# Count the occurrences of 92
```

```
print("Count of 92:", scores.count(92))
```

```
# Take input from the user
```

```
num = int(input("Enter a number: "))
```

```
# Check if the number exists in the list
if num in scores:
    print(num, "exists in the list.")
    print("Index:", scores.index(num))
    print("Count:", scores.count(num))
else:
    print(num, "does not exist in the list.")
```

""Explanation in Comments""

```
# index(value) returns the index of the first occurrence of the value.
# count(value) returns how many times the value appears in the list.
# The 'in' operator checks whether an element exists in the list.
# If the entered number is present, its first index and total count are displayed.
```

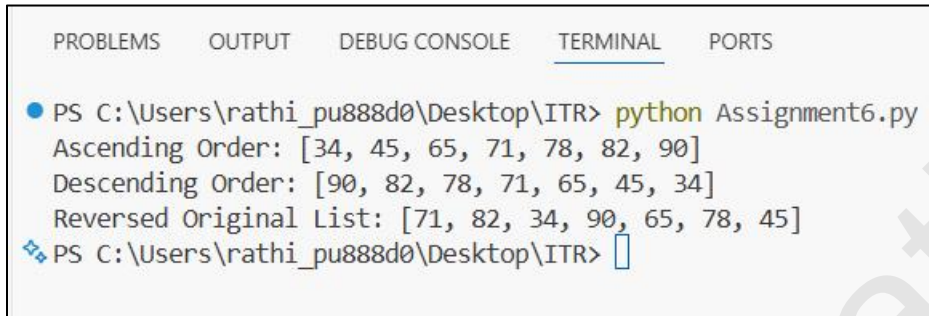
---

---

Q8. sort() and reverse()

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
Ascending Order: [34, 45, 65, 71, 78, 82, 90]
Descending Order: [90, 82, 78, 71, 65, 45, 34]
Reversed Original List: [71, 82, 34, 90, 65, 78, 45]
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

```
Q8. (sort() and reverse())
```

```
Create a list:
```

```
marks = [45, 78, 65, 90, 34, 82, 71]
```

```
Sort the list in ascending order and print it.
```

```
Sort the list in descending order and print it.
```

```
Reverse the original list (without sorting) and print it.
```

```
Add comments on the difference between sort() and reverse().
```

```
'''
```

```
# Creating the original list
```

```
marks = [45, 78, 65, 90, 34, 82, 71]
```

```
# Sort the list in ascending order
```

```
ascending_marks = marks.copy()
```

```
ascending_marks.sort()
```

```
print("Ascending Order:", ascending_marks)
```

```
# Sort the list in descending order
```

```
descending_marks = marks.copy()
```

```
descending_marks.sort(reverse=True)
```

```
print("Descending Order:", descending_marks)
```

```
# Reverse the original list without sorting
reversed_marks = marks.copy()
reversed_marks.reverse()
print("Reversed Original List:", reversed_marks)
```

"Explanation in Comments"

```
# sort() arranges the elements in ascending order by default.
# sort(reverse=True) arranges the elements in descending order.
# reverse() only changes the order of elements.
# reverse() does not sort the list.
```

```
# Difference between sort() and reverse():
# sort() organizes elements based on their values.
# reverse() simply flips the existing order of elements.
# copy() is used to keep the original list unchanged.
```

---

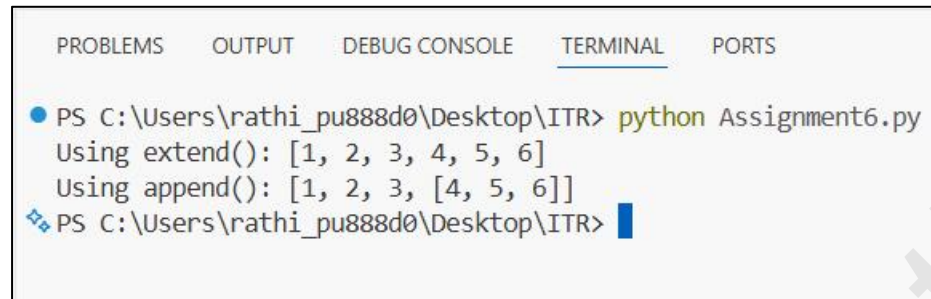


---

Q9. extend() vs append()

---

**Output Screenshot:**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
Using extend(): [1, 2, 3, 4, 5, 6]
Using append(): [1, 2, 3, [4, 5, 6]]
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

**Program Code:**

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q9. (extend() vs append())

Create two lists:

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

Demonstrate:

a) Using extend() to add list2 to list1.

b) Using append() to add list2 to a copy of list1.

Print both results and explain the difference in comments.

```
'''
```

```
# Creating the lists
```

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
# Using extend()
```

```
extend_list = list1.copy()
```

```
extend_list.extend(list2)
```

```
print("Using extend():", extend_list)
```

```
# Using append()
append_list = list1.copy()
append_list.append(list2)

print("Using append():", append_list)
```

**"Explanation in Comments"**

```
# extend() adds each element of list2 individually to the end of list1.
# append() adds the entire list2 as a single element.
```

```
# Difference between extend() and append():
# extend() merges the elements of two lists.
# append() inserts the second list as one nested list element.
```

```
# copy() is used so that the original list remains unchanged.
```

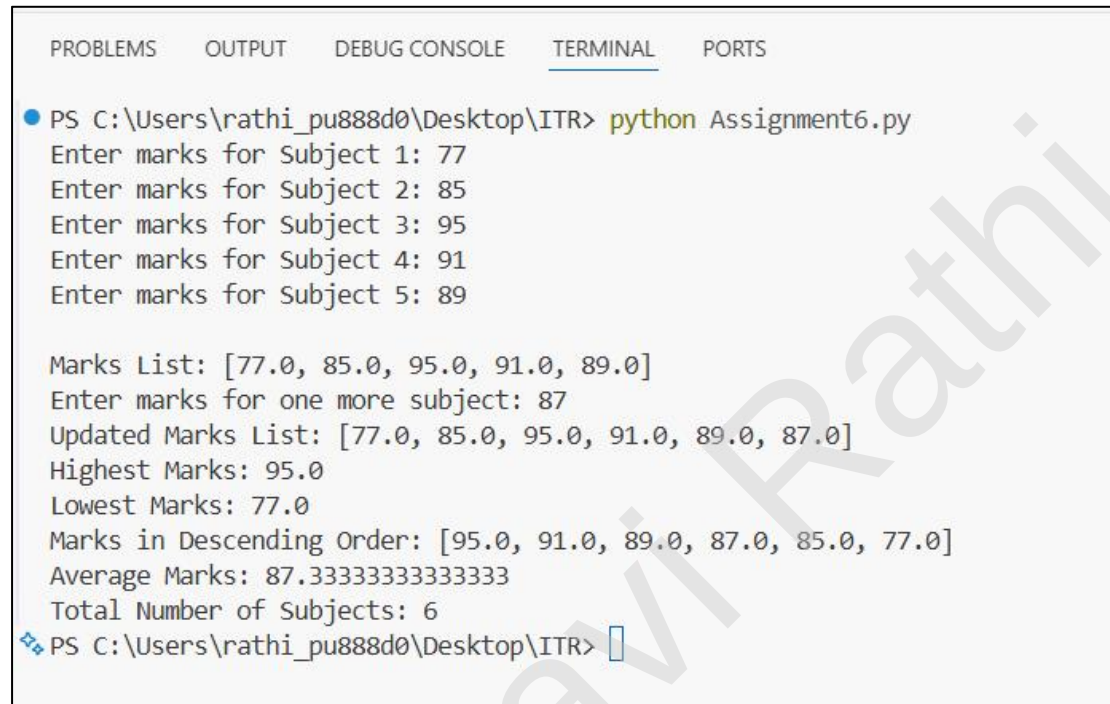
---

---

## Q10. Mini Project - Combined Concepts

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment6.py
Enter marks for Subject 1: 77
Enter marks for Subject 2: 85
Enter marks for Subject 3: 95
Enter marks for Subject 4: 91
Enter marks for Subject 5: 89

Marks List: [77.0, 85.0, 95.0, 91.0, 89.0]
Enter marks for one more subject: 87
Updated Marks List: [77.0, 85.0, 95.0, 91.0, 89.0, 87.0]
Highest Marks: 95.0
Lowest Marks: 77.0
Marks in Descending Order: [95.0, 91.0, 89.0, 87.0, 85.0, 77.0]
Average Marks: 87.33333333333333
Total Number of Subjects: 6
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

---

### Program Code:

```
""" ASSIGNMENT 6 """
```

```
'''
```

Q10. (Mini Project - Combined Concepts)

Create a Student Marks Manager program using lists:

Take 5 subject marks as input from the user and store them in a list.

Print the list.

Add one more subject mark using append().

Print the highest and lowest marks (using built-in functions).

Sort the marks in descending order.

Calculate and print the average marks.

Use len() to show total number of subjects.

Use proper comments, meaningful variable names, and control structures where needed.

```
'''
```

```

# Create an empty list to store marks
subject_marks = []

# Take 5 subject marks as input from the user
for i in range(5):
    mark = float(input(f"Enter marks for Subject {i + 1}: "))
    subject_marks.append(mark)

# Print the list of marks
print("\nMarks List:", subject_marks)

# Add one more subject mark
additional_mark = float(input("Enter marks for one more subject: "))
subject_marks.append(additional_mark)

print("Updated Marks List:", subject_marks)

# Print highest and lowest marks
print("Highest Marks:", max(subject_marks))
print("Lowest Marks:", min(subject_marks))

# Sort marks in descending order
subject_marks.sort(reverse=True)
print("Marks in Descending Order:", subject_marks)

# Calculate average marks
average_marks = sum(subject_marks) / len(subject_marks)
print("Average Marks:", average_marks)

# Print total number of subjects
print("Total Number of Subjects:", len(subject_marks))

"Explanation in Comments"

# subject_marks is a list used to store subject marks.
# append() adds new marks to the end of the list.
# max() finds the highest mark in the list.
# min() finds the lowest mark in the list.
# sort(reverse=True) arranges marks from highest to lowest.
# sum() calculates the total of all marks.
# len() returns the total number of subjects.
# Average is calculated by dividing the total marks by the number of subjects.
# A for loop is used to take marks input repeatedly.

```

---